Software Architecture and Design Specification (SAD)

AI-Powered Content Management System (AICMS)

Version: 1.0 | Date: 09-10-2026 | Status: Stage 1 – Proposed design / planning

Team / Author: Shreeverdhan M | SRN: PES1UG24AM265
Sripada Krishna Sameer Aditya | SRN: PES1UG24AM285
Srivamshi Chikkireddi | SRN: PES1UG24AM287
Shamanth SK | SRN: PES1UG24AM258

Submission boundary: This submission defines requirements, proposed design, and planned verification. It does not claim that a working implementation exists or that tests have been executed.

Revision History

Version	Date	Author	Change Summary	Approval
1.0	09-10-2026	AICMS Team (Shreeverdhan M, Sripada Krishna Sameer Aditya, Srivamshi Chikkireddi, Shamanth SK)	Initial proposed architecture and design for AICMS	Pending

Approvals

Role	Name	Signature / Date
Course Coordinator		
Team Representative		

1. Introduction

1.1 Purpose

This document specifies the proposed logical architecture and design of AICMS and maps its components and interfaces to the accompanying SRS.

1.2 Scope

The design covers document intake, text extraction/OCR, AI categorization and keyword/topic extraction, embeddings, indexing, natural-language search, metadata correction, access control, audit, and monitoring. This is a Stage 1 design; no deployed or working implementation is claimed.

1.3 Audience

Project evaluators, analysts, developers, QA engineers, security reviewers, administrators, and maintenance teams.

1.4 Definitions

AICMS – AI-Powered Content Management System; NLP – Natural Language Processing; OCR – Optical Character Recognition; RBAC – Role-Based Access Control; ADR – Architecture Decision Record; API – Application Programming Interface; embedding – vector representation of text.

2. Document Overview

2.1 How to Use This Document

Read with the SRS and Software Test Plan. Component/class diagrams define logical responsibilities and domain objects; sequence diagrams define representative interaction flows; API contracts and ADRs document proposed design choices. These artifacts describe intended behavior, not completed software.

2.2 Related Documents

AICMS Software Requirements Specification (SRS), AICMS Software Test Plan (STP), and the SRS Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals and Constraints

- Provide useful natural-language retrieval while preventing unauthorized disclosure through search snippets or metadata.
- Separate parsing, AI inference, storage, indexing, and user interaction so each can be tested and changed independently.
- Keep long parsing/model jobs asynchronous and observable with clear retry/failure status.
- Support metadata correction, deterministic re-indexing, deletion propagation, and traceability.
- Constraints: model latency and cost, document format/language variability, OCR quality, privacy/data-residency policy, and index consistency.
- Availability and performance values are proposed targets requiring future measurement; they are not achieved claims.
- Proposed quality targets (see SRS section 5): search p90 ≤ 3 s; at least 80% correct categorization on the labelled test set; processing of a 10-page document within 60 s.

3.2 Stakeholders and Concerns

Stakeholder	Primary concerns	Architectural response
General user	Fast, relevant search; clear upload status; easy navigation.	Natural-language search API, filters, status UI, readable snippets.
Content editor	Correct AI suggestions and control over metadata.	Editable metadata with validation, permissions, and audit.
Administrator	Access control, taxonomy, operational status, re-indexing.	Admin functions, job visibility, metrics, audit trail.
Data owner / security reviewer	Confidentiality, retention, approved AI data processing.	Permission filtering, secure transport/secrets, approved model policy.
Developer / QA	Clear interfaces, testability, recoverability.	Layered modules, async jobs, APIs, requirement/test traceability.

3.3 UML Component Diagram

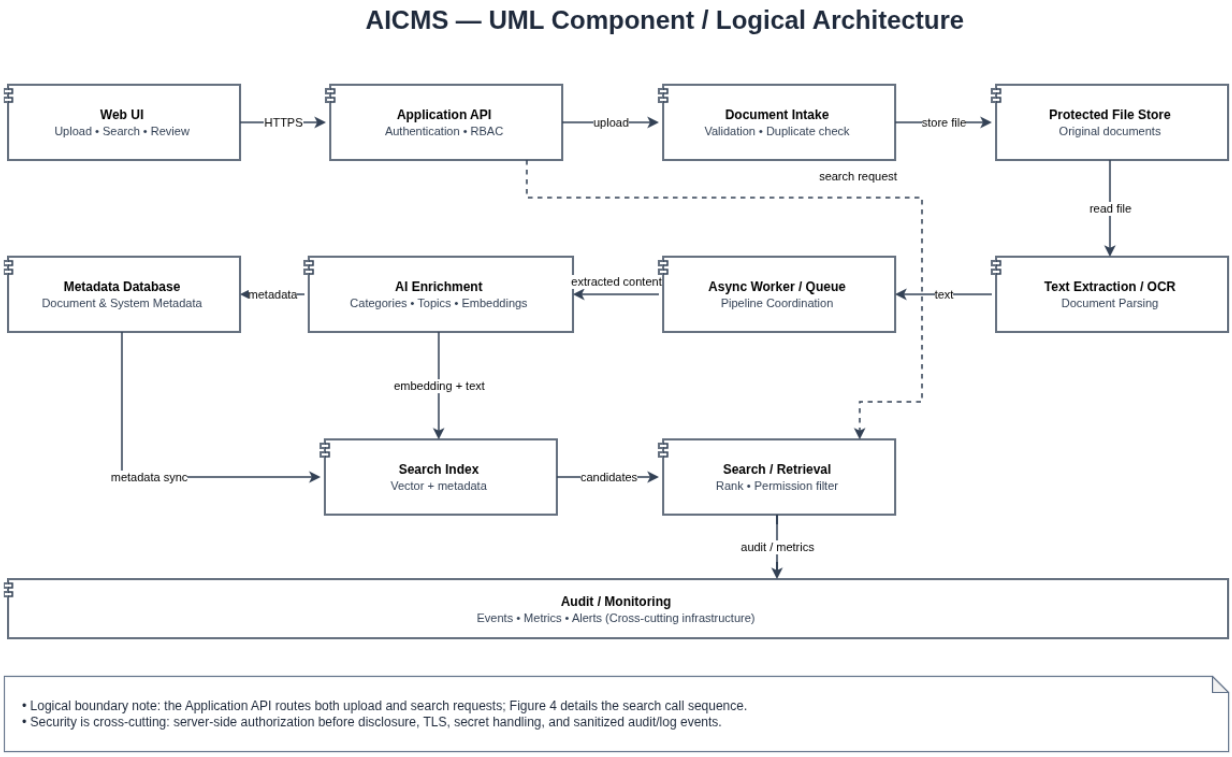


Figure 1. Proposed logical components and major data/control paths. Cross-cutting audit and monitoring apply across components.

3.3.1 UML Class Diagram

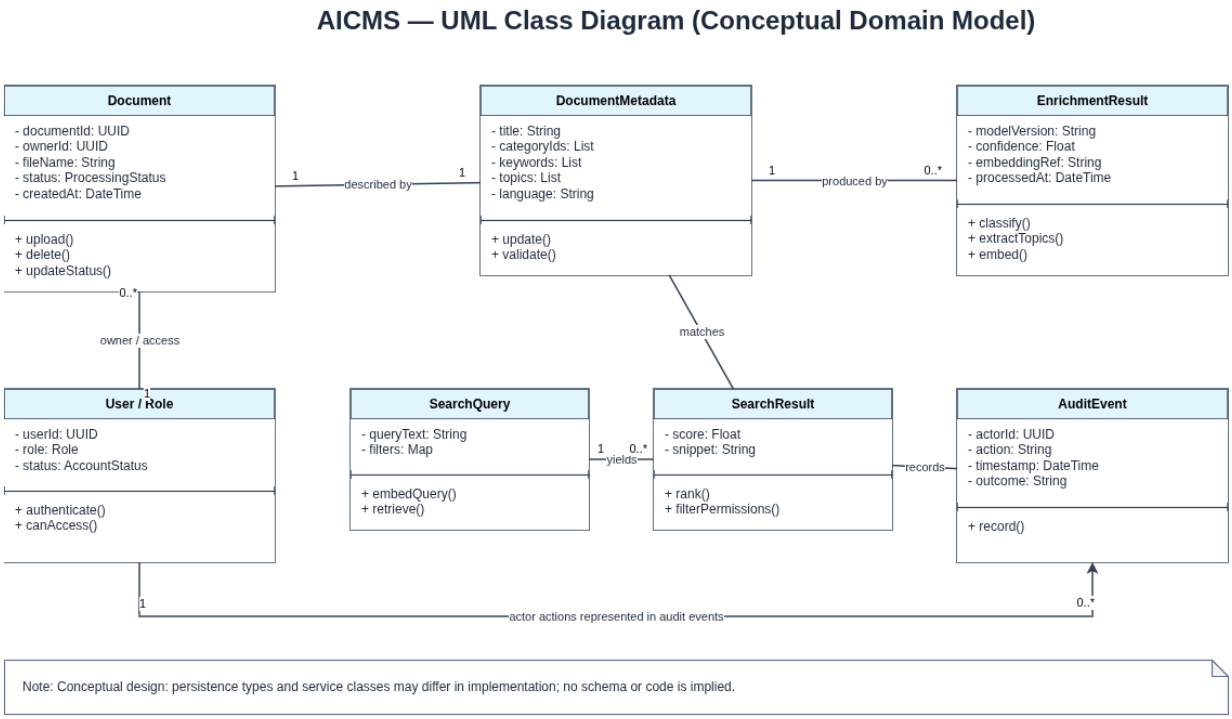


Figure 2. Conceptual domain model; actual persistence classes and schema remain implementation decisions.

3.4 Component Descriptions

Component	Responsibilities	Inputs / outputs and key controls
Web UI	Upload, search, result browsing, metadata editing, and admin screens.	User actions and authorized results; accessibility and clear status/errors.
Application API	Validate request shape, authenticate identity, authorize operations, orchestrate services, rate-limit.	HTTPS API requests/responses; safe errors and correlation ID.
Document Intake	Check type/size/content signature, apply duplicate policy, assign document ID.	Uploaded file → stored file + processing record; untrusted content handled defensively.
Document/Object Storage	Store original files separately from structured metadata.	Document ID → protected file; permissions and retention policy apply.
Text Extraction / OCR	Extract text and common metadata from supported formats and optional scans.	File → text, metadata, quality flags; parser execution isolated where possible.
Async Queue / Worker	Schedule extraction, enrichment, re-index, retry, and deletion propagation.	Job IDs/statuses; bounded retries and reviewable terminal errors.
AI Enrichment Pipeline	Suggest categories, extract keywords/topics, create document embeddings.	Text → versioned enrichment output and embedding; output treated as a suggestion.
Metadata Service / DB	Persist ownership, metadata, processing state, category references, and audit references.	Validated CRUD operations and data integrity constraints.
Search Index	Maintain lexical/vector representations and filterable metadata.	Document ID, text, vector, filters → candidates and relevance scores.
Search / Retrieval API	Embed query, retrieve candidates, apply authorization filters before formatting, rank results.	Natural-language query → authorized ranked results and snippets.
Audit / Observability	Capture security/admin events and aggregate latency/error/queue metrics.	Sanitized event and metric stream; no secrets or unnecessary raw document text.

3.5 Chosen Architecture Pattern and Rationale

Proposed pattern: layered modular architecture with an asynchronous document-processing pipeline. The UI, API, domain functions, worker pipeline, persistence, and search index have explicit responsibilities and interfaces. This lowers coupling and makes it possible to replace a parser, model, or search engine without redesigning user workflows. A full microservices deployment is not justified at Stage 1; modules may initially run in one deployment while preserving logical boundaries.

3.6 Technology Stack and Data Stores

Area	Proposed option (recommended default)	Rationale / selection gate
Frontend	Responsive browser UI — recommended default: React (plain HTML/JavaScript is acceptable for a prototype).	Accessible upload/search/editor flows; final choice follows team skills.
Backend API	Python with FastAPI (recommended default); Java/Spring Boot is an alternative.	Python matches the AI/NLP libraries; confirm after a short prototype.
Metadata store	PostgreSQL (SQLite acceptable for a prototype).	Transactions and structured metadata; schema review required.
Document storage	Protected object or file storage.	Separate binary documents from metadata; retention/backup policies needed.
Search	Vector search with metadata filters — e.g., PostgreSQL with pgvector, or OpenSearch/Elasticsearch with vector support; cosine similarity between query and document embeddings.	Support semantic similarity, filtering, and optional lexical (keyword) hybrid retrieval.
AI / NLP	Pre-trained open-source sentence-embedding model (e.g., all-MiniLM-L6-v2); scikit-learn classifier on the embeddings; TF-IDF/key-phrase extraction and a topic model (LDA/NMF).	Evaluate quality, latency, language coverage, cost, and data-handling terms; runs without a GPU.
Async processing	Queue + worker pattern — e.g., Celery with Redis, or a simple background job runner for a prototype.	Retryable extraction/enrichment, status tracking, re-indexing.
Security / operations	HTTPS/TLS, RBAC, secret store, structured logs and metrics.	Required for access control, diagnosis, and operational evidence.

3.7 Risks and Mitigations

Risk	Potential impact	Mitigation / verification
Incorrect category/topic suggestions	Poor organization and reduced trust.	Treat output as a suggestion; allow edits; evaluate against labelled samples.
Low-quality extraction / OCR	Missing content and weak search.	Flag extraction confidence/quality; test varied files; enable review/retry.
Permission leakage in search	Confidentiality breach via title/snippet/metadata.	Filter candidates by authorization before formatting; cross-role negative tests.
Model or parser service outage	Incomplete processing and queue growth.	Asynchronous jobs, bounded retry/backoff, visible failure state, operational metrics.
Stale index after edit/delete	Search returns obsolete or deleted content.	Idempotent events, index version/state, reconciliation and deletion tests.
Unapproved external AI processing	Data privacy/compliance risk.	Data classification and provider approval gate; use approved hosting and minimize payloads.

3.8 Traceability to Requirements

SRS requirement group	Architecture component(s)	Design rationale / verification
AICMS-F-001–004 (intake/status)	Web UI, API, Document Intake, storage, job queue.	Validate before processing; track status and duplicate policy.
AICMS-F-005–010 (extract/enrich)	Extraction/OCR, Worker, AI Enrichment, metadata DB.	Keep processing asynchronous and record model/error state.
AICMS-F-011–015 (index/search)	Search Index, Search API, Web UI.	Rank authorized candidates; test search quality, filters, snippets.

SRS requirement group	Architecture component(s)	Design rationale / verification
AICMS-F-016–018, 023; AICMS-SR-001, 004, 006–008	Metadata Service, RBAC, Search API, Audit.	Authorization before disclosure; audit changes/access.
AICMS-F-019–022, 024 (admin/re-index/delete/metrics/users)	Admin UI, Worker, Index, Observability.	Queue-controlled changes, consistency checks, metrics.
AICMS-NF-001–010; AICMS-SR-002, 003, 005	API, job workers, secure configuration, monitoring.	Performance, safe uploads, TLS, secrets/logging, failure states.

3.9 Security Architecture and Threat Model

Threats are reviewed using the STRIDE categories. Controls below are design proposals and must be verified during implementation and security testing.

STRIDE threat	AICMS example	Proposed mitigations / verification
Spoofing	Attacker impersonates a user or administrator.	Institution-approved authentication, secure session/token handling, role checks; test unauthorized access. Salted password hashing, 30-minute session expiry and login lock-out after 5 failures (AICMS-SR-007, SR-008).
Tampering	Uploaded file, metadata, or index record is maliciously changed.	Validate inputs, controlled writes, checksums/versioned jobs, audit important changes.
Repudiation	User denies metadata deletion or role change.	Timestamped audit events with actor/action/outcome; protect audit write path and retention.
Information disclosure	Restricted file title/snippet/vector leaks from semantic results or logs.	Authorization before formatting results; TLS; least privilege; secret/raw-content log exclusion.
Denial of service	Oversized uploads, expensive queries, queue flooding.	Size limits, validation, rate limits, bounded concurrency, timeouts and queue monitoring.
Elevation of privilege	Normal editor invokes administrator or cross-document operations.	RBAC + document-level authorization at server-side APIs; negative tests for every protected path.

4. Design

4.1 Design Overview

Upload flow: the API authenticates and validates the request, the intake component stores the original and queues a job, and the worker extracts content, performs AI enrichment, and updates metadata/index status. Search flow: the API validates the query, retrieval creates a query vector if required, candidates are retrieved, permission checks are applied before snippets are assembled, and authorized results are ranked and returned. Failed or partial jobs remain visibly failed/retryable rather than appearing complete.

4.2 UML Sequence Diagrams

4.2.1 Upload, Enrichment, and Indexing

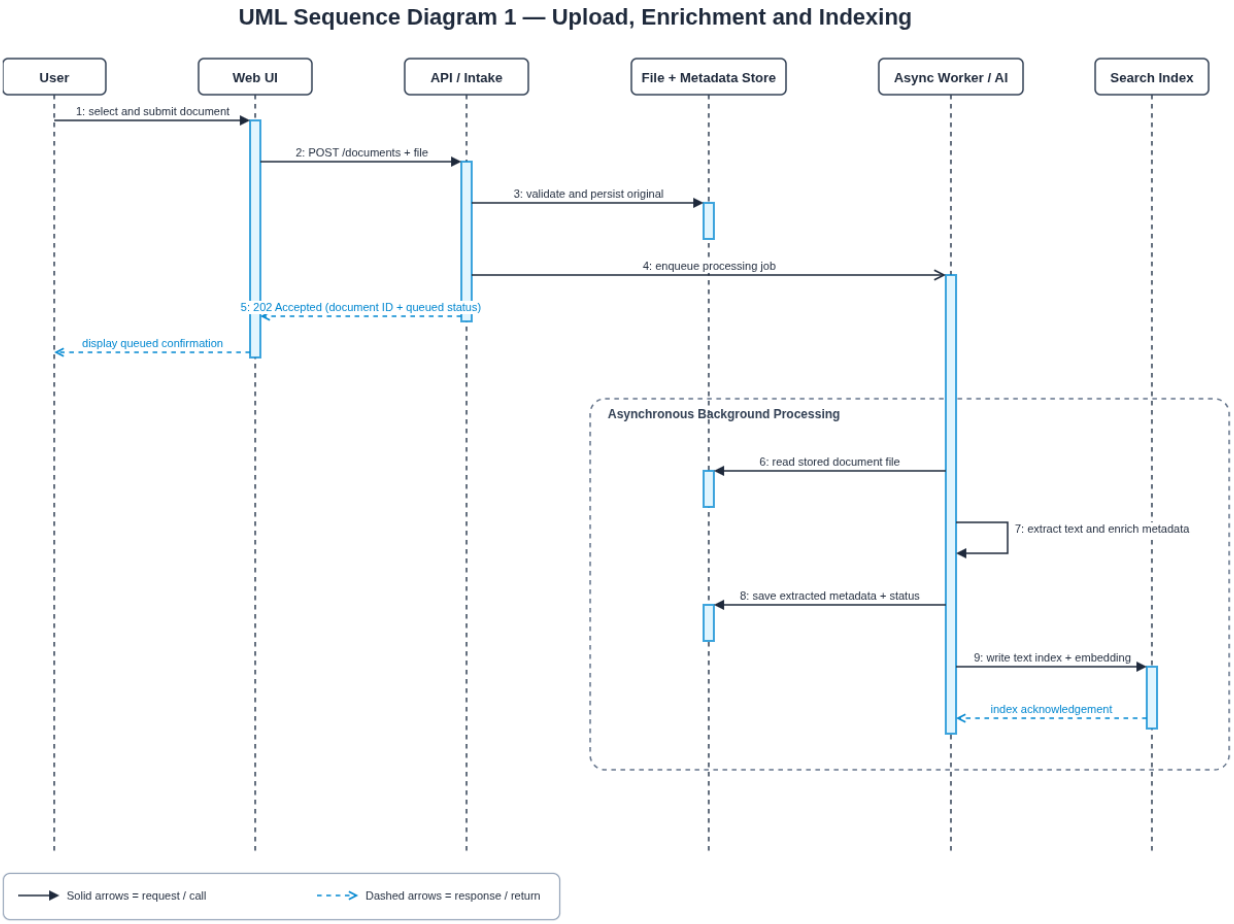


Figure 3. Solid arrows show calls; dashed arrows show responses. Processing continues asynchronously after the upload request is accepted.

4.2.2 Natural-language Semantic Search

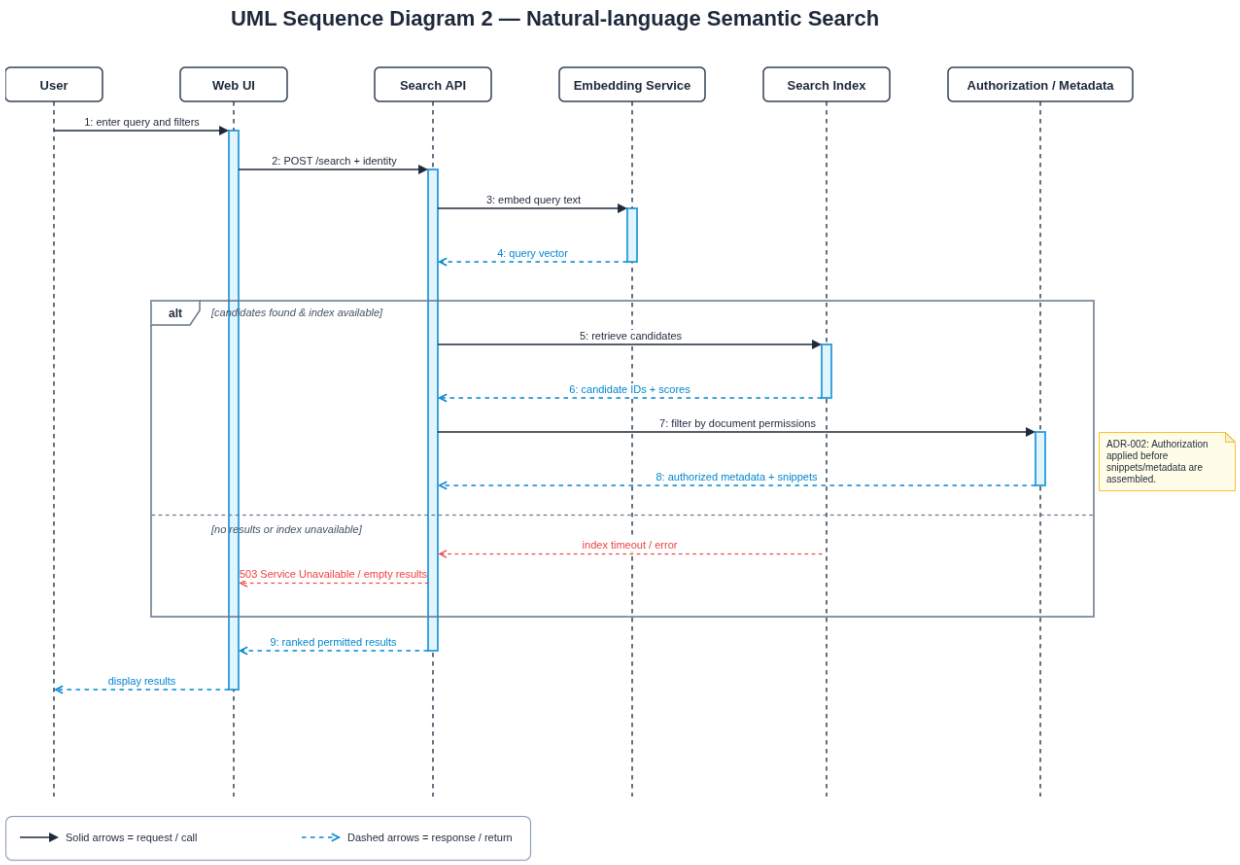


Figure 4. Authorization is applied to retrieved candidates before protected metadata/snippets are returned.

4.3 API Design

The following REST-style endpoint contracts are proposed. Exact request validation, identity claims, pagination defaults, and response schemas must be confirmed before implementation.

Component / endpoint	Method	Request / purpose	Successful response	Representative errors
Document Intake: /api/v1/documents	POST	Multipart file; authenticated identity; optional metadata.	202 Accepted: document_id, status=QUEUED.	400 invalid input; 401 unauthenticated; 403 forbidden; 413 too large; 409 duplicate per policy.
Document Query: /api/v1/documents/{id}	GET	Read one permitted document's metadata/status.	200: document_id, status, authorized metadata.	401 unauthenticated; 403 forbidden; 404 missing/not visible.
Search API: /api/v1/search	POST	JSON {query, filters, page, page_size}.	200: results[], total, query_time_ms.	400 invalid query; 401/403 unauthorized; 429 rate limit; 503 index unavailable.
Metadata Service: /api/v1/documents/{id}/metadata	PATCH	Editable category/keywords/topics; editor authorization.	200 updated metadata and revision.	400 invalid fields; 403 forbidden; 404 not found; 409 revision conflict.
Admin /api/v1/admin/reindex	POST	Authorized scope and model/taxonomy version.	202: job_id, status=QUEUED.	400 invalid request; 403 forbidden; 409 conflicting job.
Document deletion: /api/v1/documents/{id}	DELETE	Authenticated owner/editor/admin subject to policy.	202 deletion job/status; final state removes index entries.	401 unauthenticated; 403 forbidden; 404 missing/not visible.
Authentication: /api/v1/auth/login	POST	JSON {username, password} (or identity-provider token exchange).	200: session token with 30-minute idle expiry.	400 invalid input; 401 invalid credentials (generic message); 423 account temporarily locked; 429 rate limit.
Admin users: /api/v1/admin/users/{id}/role	PUT	JSON {role}; administrator only; also create/deactivate via /api/v1/admin/users.	200: user_id, role, revision.	400 invalid role; 403 forbidden; 404 not found; 409 revision conflict.

4.4 Error Handling, Logging, and Monitoring

- Use a consistent safe error body with stable error code, short message, and correlation ID; never send stack traces to users.
- Do not log passwords, tokens, raw embeddings, or unnecessary document text; use document IDs and sanitized metadata where needed.
- Track upload acceptance/failure, parser/OCR duration, AI job duration, queue depth, search p50/p90 latency, error rate, empty-result rate, and index reconciliation exceptions.
- Retry only transient failures with bounded backoff; keep terminal failures reviewable and make retries idempotent.
- Define operational alert thresholds, backup/restore, retention, and incident procedures during implementation planning.

4.5 UX Design

- Use one prominent natural-language search input with optional, understandable filters.
- Show processing status and actionable file validation or processing messages.
- Label categories/topics as AI-generated suggestions; provide authorized editor review.
- Preserve visible keyboard focus, accessible labels, high contrast, and responsive layouts.
- Show safe empty/error states and avoid guaranteeing that semantic matches are always correct.

4.6 Open Issues and Next Steps

- Confirm file formats, maximum file size, supported languages, OCR need, and duplicate policy.
- Validate the default embedding model and search engine on a representative labelled corpus before release (ADR-004).
- Approve deployment location and data-sharing rules for any external model service.
- Agree retention, deletion, backup/recovery, and audit-retention policy.
- If Stage 2 is approved, build a proof of concept, implement the API contracts, and execute the test plan before release.

5. Appendices

5.1 Glossary

AICMS – AI-Powered Content Management System; ADR – Architecture Decision Record; API – Application Programming Interface; embedding – vector representation of text; NLP – Natural Language Processing; OCR – Optical Character Recognition; RBAC – Role-Based Access Control; RTM – Requirements Traceability Matrix; STRIDE – Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.

5.2 References

- ISO/IEC/IEEE 42010, architecture description (reference framework).
- OWASP Application Security Verification Standard and File Upload guidance.
- NIST SP 800-160, systems security engineering (reference framework).
- WCAG 2.1 guidance for applicable accessibility requirements.

5.3 Tools

Suggested design tools: draw.io or PlantUML for editable diagram sources, a word processor for specifications, and OpenAPI/Swagger for API contract documentation after schemas are agreed.

5.4 Architecture Decision Records (ADRs)

ADR	Decision / status	Context and rationale	Consequences / revisit condition
ADR-001	Use a layered modular architecture with asynchronous processing — Proposed.	Document parsing and AI inference have variable duration; clear module boundaries support testing and technology replacement.	Requires job status, retries, idempotency, and monitoring. Revisit if scale/team boundaries justify deployment separation.
ADR-002	Enforce authorization before search results/snippets are formatted — Required design constraint.	Semantic candidate retrieval must not leak restricted titles, snippets, vectors, or metadata.	Search pipeline must have identity/permission context and negative access tests. Revisit only with an equivalent provably safe design.
ADR-003	Treat AI-enriched metadata as editable suggestions — Proposed.	Model predictions vary by corpus, language, and model; incorrect labels should not silently become authoritative facts.	Store review/correction state and test edits/re-indexing. Revisit if validated automation and governance policy permit automatic final labels.
ADR-004	Adopt the default stack in section 3.6 (React, FastAPI, PostgreSQL with pgvector, Celery/Redis, all-MiniLM-L6-v2 with a scikit-learn classifier) — Accepted.	Python matches the AI/NLP libraries used for enrichment, the stack runs without a GPU, and PostgreSQL with pgvector covers vector search with metadata filters in one database.	Validate against AICMS-NF-001, NF-009 and NF-010 on the labelled corpus; if a target is missed, record the replacement as a new ADR.